<Bolsa Futuro Digital>

<aula 26>

Turma de FrontEnd - IF UFRGS

Escopo

Escopo é a região do código onde uma variável pode ser acessada. Cada variável tem um alcance — fora dele, ela simplesmente não existe.

Existem três tipos de escopo:

- **Global** — acessível em qualquer lugar
- **de Função** — só existe dentro da função
- **de Bloco** — só existe dentro do { }

```
let nome = 'Ana'              // global

function saudacao() {
  let mensagem = 'Olá!'       // local
  console.log(nome)           // ✓ acessa global
  console.log(mensagem)       // ✓ acessa local
}

saudacao()

console.log(nome)             // ✓ 'Ana'
console.log(mensagem)         // ✗ ReferenceError!
                              // mensagem não existe aqui
```

Escopo de bloco

```
// var ignora blocos {}

if (true) {
  var cidade = 'Porto Alegre'
}
console.log(cidade)
// 'Porto Alegre' ← vazou!

for (var i = 0; i < 3; i++) {
  // ...
}
console.log(i)  // 3 ← ainda existe!

// Use var é imprevisível fora
// do escopo que você imaginou.
```

```
// let e const respeitam blocos {}

if (true) {
  let cidade = 'Porto Alegre'
}
console.log(cidade)
// ✗ ReferenceError!

for (let j = 0; j < 3; j++) {
  // ...
}
console.log(j)  // ✗ ReferenceError!

// let e const se comportam
// como o esperado. Prefira-os!
```

Escopo de função

Variáveis declaradas dentro de uma função **existem apenas ali**. Cada chamada cria um escopo separado — como uma **sala isolada**.

Isso vale para **var, let e const**. Nem mesmo o var vaza para fora de uma função.

Cada chamada à função tem seu **próprio conjunto** de variáveis — independente das outras chamadas.

```
function calcular(a, b) {
  let resultado = a + b   // existe só aqui
  console.log(resultado)  // ✓ 15
}

calcular(10, 5)
console.log(resultado)    // ✗ ReferenceError!

// Cada chamada tem seu próprio escopo:
function cumprimentar(nome) {
  let msg = 'Olá, ' + nome + '!'
  console.log(msg)
}

cumprimentar('Ana')   // Olá, Ana!
cumprimentar('João')  // Olá, João!
// cada chamada tem seu próprio 'msg'
```

Closure

Uma **closure** acontece quando uma função lembra do escopo onde foi criada, mesmo depois que esse escopo terminou.

A função carrega consigo as variáveis do ambiente externo — como uma mochila de memória.

Closures acontecem automaticamente no JavaScript — você não precisa fazer nada especial para ativá-las.

```
function criarSaudacao(nome) {
  // 'nome' fica guardado na closure

  return function() {
    console.log('Olá, ' + nome + '!')
  }
}

let saudarAna  = criarSaudacao('Ana')
let saudarJoao = criarSaudacao('João')

saudarAna()   // Olá, Ana!
saudarJoao()  // Olá, João!

// criarSaudacao já terminou de executar,
// mas 'nome' ainda está acessível!
// Isso é uma closure.
```

Closure

O uso mais clássico de closure é criar **estado privado.** A variável interna fica protegida, ninguém de fora consegue acessá-la ou modificá-la diretamente.

Cada chamada a **criarContador()** cria um escopo **independente,** dois contadores nunca interferem entre si.

```
function criarContador() {
  let total = 0          // variável privada!

  return function() {
    total = total + 1
    return total
  }
}

let contar = criarContador()

console.log(contar())  // 1
console.log(contar())  // 2
console.log(contar())  // 3

// Contadores independentes:
let contarCliques = criarContador()
let contarErros   = criarContador()

contarCliques()  // 1
contarCliques()  // 2
contarErros()    // 1 ← independente!
```

Funções como valor

Em JavaScript, funções são valores como qualquer outro, você pode guardá-las em variáveis, **passá-las como argumento** e retorná-las de outras funções.

Isso permite criar código mais flexível e reutilizável. Esse conceito é a base dos **callbacks**.

Atenção: ao passar uma função, não use parênteses.

```
// Função guardada em variável
let somar = function(a, b) {
  return a + b
}

console.log(somar(3, 4))   // 7
console.log(typeof somar)   // 'function'

// Passando função como argumento
function executar(fn) {
  console.log('Executando...')
  fn()
}

function dizerOi() {
  console.log('Oi!')
}

executar(dizerOi)   // ✓ passa a função
// Executando...
// Oi!

executar(dizerOi()) // ✗ passa o resultado (undefined)
```

Callbacks

Um **callback** é uma função passada como argumento para ser chamada em um momento específico.

É como deixar um número de telefone: "me liga quando terminar".

Já usamos callbacks o tempo todo com **addEventListener** — passamos uma função para ser chamada quando o evento acontece.

```
// addEventListener já é um callback!
let btn = document.getElementById('btn')

btn.addEventListener('click', function() {
  console.log('Clicado!')  // ← callback
})

// Criando sua própria função com callback:
function processar(dados, callback) {
  console.log('Processando: ' + dados)
  callback(dados)
}

function exibir(valor) {
  console.log('Resultado: ' + valor)
}

processar('relatório', exibir)
// Processando: relatório
// Resultado: relatório
```

Callbacks

Callbacks permitem delegar a decisão de o que fazer com o resultado. A função que recebe o callback não precisa saber o que vai acontecer — ela só executa e avisa.

Esse padrão torna as funções mais genéricas e reutilizáveis.

```
function executarSe(condicao, callback) {
  if (condicao) {
    callback()
  }
}

executarSe(true, function() {
  console.log('Condição verdadeira!')
})
// Condição verdadeira!

executarSe(false, function() {
  console.log('Isso não roda')
})
// (nada é exibido)

// Validação com callback:
function validar(valor, aoValidar) {
  if (valor === '') {
    aoValidar(false, 'Campo obrigatório')
  } else {
    aoValidar(true, 'Válido!')
  }
}
```

Referência vs Valor

Tipos **primitivos** (number, string, boolean, null, undefined) são copiados por valor — cada variável recebe uma cópia independente.

Modificar uma variável não afeta a outra. Elas são completamente separadas na memória.

```
// Primitivos: cópia por VALOR
let a = 10
let b = a          // b recebe uma CÓPIA de a

b = 20

console.log(a)     // 10 ← não mudou!
console.log(b)     // 20

// O mesmo com strings:
let nome1 = 'Ana'
let nome2 = nome1   // cópia independente

nome2 = 'João'

console.log(nome1)  // 'Ana' ← intacto!
console.log(nome2)  // 'João'

// Comparação usa o VALOR:
let x = 5
let y = 5
console.log(x === y)  // true ← mesmos valores
```

Referência vs Valor

Objetos e arrays não são copiados — apenas a referência (o endereço na memória) é copiada. Duas variáveis passam a apontar para o mesmo objeto!

Isso causa bugs difíceis de perceber: você modifica uma variável e outra "muda sozinha".

```
// Objetos: cópia por REFERÊNCIA
let usuario1 = { nome: 'Ana', idade: 20 }
let usuario2 = usuario1  // ← mesma referência!

usuario2.nome = 'João'

console.log(usuario1.nome)  // 'João' ← mudou!
console.log(usuario2.nome)  // 'João'

// O mesmo com arrays:
let notas1 = [7, 8, 9]
let notas2 = notas1  // ← mesma referência!

notas2.push(10)

console.log(notas1)  // [7, 8, 9, 10] ← mudou!
console.log(notas2)  // [7, 8, 9, 10]

// São o mesmo objeto na memória:
console.log(usuario1 === usuario2)  // true
```

Copiando objetos

```
// Abordagem 1: JSON (visto na Aula 25)
// stringify → string
// parse     → objeto NOVO na memória

let original = { nome: 'Ana', nota: 9 }

let copia = JSON.parse(
  JSON.stringify(original)
)

copia.nome = 'João'

console.log(original.nome)  // 'Ana' ✓
console.log(copia.nome)     // 'João'

// Objetos separados:
console.log(original === copia)  // false ✓
```

```
// Abordagem 2: copiar com for...in

let produto = { nome: 'Teclado', preco: 350 }
let novoProduto = {}

for (let chave in produto) {
  novoProduto[chave] = produto[chave]
}

novoProduto.preco = 400

console.log(produto.preco)      // 350 ✓
console.log(novoProduto.preco)   // 400

// Objetos separados:
console.log(produto === novoProduto)  // false ✓
```

Exercícios

Mini projeto: O objetivo é montar uma pequena página onde o usuário cria fichas de personagem de RPG. Cada ficha tem um nome, uma classe (Guerreiro, Mago ou Ladino) e quatro atributos: Força, Agilidade, Inteligência e Resistência. O usuário distribui pontos entre os atributos, pode receber um bônus de acordo com a classe escolhida e, no final, salva a ficha, que aparece numa lista logo abaixo.
Mais do que fazer a página funcionar, o exercício serve para praticar três conceitos importantes de JavaScript: closures, callbacks e a diferença entre referência e cópia de objetos.

Exercícios


Ficha de Personagem
Nome
Guerreiro
Pontos: 10
Força
5
Agilidade
5
Inteligência
5
Resistência
5
Aplicar Bônus
Salvar Ficha
© GabrielScheffel

</aula 26>